Lab 09 — Hands-on lab: the stack in one file

Goal: start the complete `web → api → db` stack with a single file, watch the start order enforced by the healthchecks, operate the stack day to day (`logs`, `exec`, `stop`, `down`), switch between development and production with override files, and touch the limits of podman-compose.

Prerequisites — Labs 01 to 08 completed. Image `api:1.0` present, `~/labo-docker/05/Dockerfile` present. **podman-compose installed**: `sudo apt install podman-compose` (if the version shown by `podman compose version` is below 1.1, prefer `pipx install podman-compose`). The output below comes from podman-compose 1.6.0.

Files provided (`files/`) - `compose.yaml` — the stack: `db` (PostgreSQL, volume, secret, healthcheck), `api` (image or `build: ../05`, `env_file`, `depends_on`), `web` (nginx, port 18080). - `compose.override.yaml` — development additions, read automatically: database published on `127.0.0.1:5432`, `dev` profile. - `compose.prod.yaml` — production variant, to combine with `-f`: `restart: unless-stopped`, `prod` profile. - `dot-env` — to rename to `.env`: `API_TAG=1.0`, substituted into `compose.yaml`. - `api.env` — the API configuration (lab 08). - `nginx-proxy.conf`, `index.html` — the front end and reverse proxy from lab 07.

Step 0 — Setting up

```
mkdir -p ~/labo-docker/09 && cd ~/labo-docker/09
cp <path-to-the-lab>/files/* . && mv dot-env .env
printf 'S3cr3t-pg' > db_password.txt
podman compose version 2>&1 | tail -2
podman compose config | grep -E '^name:|image:|context:'
```

Observe the banner `>>>> Executing external compose provider "/usr/bin/podman-compose"` then `podman-compose version 1.6.0`; and in the merged configuration: `name: stack`, `image: localhost/api:1.0` (the `${API_TAG:-1.0}` was substituted), `context: /home/<you>/labo-docker/05`.

Explanation. `podman compose` relays to podman-compose. `config` shows what the tool understood from the YAML, override included — read it before any `up`.

Step 1 — `up`, and the start order

```
podman compose up -d
podman compose ps
```

Observe the three containers created in the order `stack_db_1`, `stack_api_1`, `stack_web_1`, and in `ps` staggered ages: `db Up 11 seconds (healthy)`, `api Up 6 seconds (healthy)`, `web Up 1 second`. Each service waited for the previous one to be `healthy`.

```
podman ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
podman network ls ; podman volume ls ; podman pod ls
```

Observe `127.0.0.1:5432->5432/tcp` on `db` (the development override was applied), `0.0.0.0:18080->80/tcp` on `web`; a `stack_default` network, a `stack_pgdata` volume, and a `pod_stack` pod with `3` containers and **no** infra.

Explanation. Everything carries the project prefix. The pod is a grouping specific to podman-compose: each service keeps its own IP and DNS name.

Step 2 — Operating the stack

```
curl -s localhost:18080/api/actuator/health ; echo ; curl -s localhost:18080/api/ ; echo
podman compose logs --tail=2 api
podman compose exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT INTO clients(nom) VALUES ('Durand');"
podman compose exec api env | grep -E 'APP_|DB_'
podman run --rm --network stack_default alpine getent hosts api db web
```

Observe `{"status":"UP"}` and `{"message":"bonjour depuis compose","profile":"dev"}` (the message comes from `api.env`, the profile from the override); logs prefixed with `[api] |`; `CREATE TABLE` / `INSERT 0 1`; the variables `DB_HOST=db`, `DB_PASSWORD_FILE=/run/secrets/db_password`; and the three names resolved to `10.89.0.x`.

Explanation. `podman compose exec` and `podman compose logs` target a **service** by name; underneath, they are `podman exec stack_db_1` and `podman logs stack_api_1`.

Step 3 — Stop, restart, and meet the `502` again

```
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
podman compose stop api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose start api ; sleep 6
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose restart web ; sleep 2
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
```

Observe `10.89.0.3`, then `502` (or `504`, depending on timing) while the API is stopped, then a **different** address after `start` (`10.89.0.6`) and still `502`: nginx keeps the old IP (lab 07). After `restart web`: `200`.

Explanation. Compose changes nothing here: a `stop`/`start` releases and reassigns the address, and nginx only resolves at startup.

Step 4 — `down` keeps the data, `down -v` does not

```
podman compose down
podman ps -a --format '{{.Names}}' ; podman network ls --format '{{.Name}}' ; podman pod ls --format '{{.Name}}' ; podman volume ls --format '{{.Name}}'
podman compose up -d ; sleep 12
podman compose exec db psql -U postgres -c "SELECT * FROM clients;"
```

Observe after `down`: no container, only the `podman` network, no pod — but `stack_pgdata` **still there**. After the second `up`, the `clients` table and `Durand` are back.

Explanation. `down` removes what is disposable; the volumes, which carry the data, require `-v`. That is Compose's safety net.

Step 5 — Development and production

```
podman compose down
podman compose -f compose.yaml -f compose.prod.yaml config | grep -E 'restart|APP_PROFILE|5432'
podman compose -f compose.yaml -f compose.prod.yaml up -d ; sleep 12
podman compose ps
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'
podman exec stack_api_1 env | grep APP_PROFILE
podman compose exec api env | grep APP_PROFILE
podman compose -f compose.yaml -f compose.prod.yaml down
```

Observe in `config`: three `restart: unless-stopped`, `APP_PROFILE: prod`, and **no** `5432` (the override is no longer read); in `ps`, `db` with no published port; `unless-stopped`; `APP_PROFILE=prod` through `podman exec`... but `APP_PROFILE=dev` through `podman compose exec` without `-f`: it re-read the override.

Explanation. `-f` is explicit and exclusive; `exec` rebuilds the environment from the files you give it, not from the container. Always pass the same `-f` (or `COMPOSE_FILE`), or use `podman exec`.

```
API_TAG=2.0 podman compose config | grep 'image: localhost'
```

Observe `image: localhost/api:2.0`: substitution from the shell takes precedence over `.env`. No `2.0` image exists — `config` does not check.

Step 6 — Two APIs, and the limit of podman-compose

```
podman compose up -d --scale api=2 ; sleep 12
podman compose ps
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address
for i in 1 2 3 4 ; do curl -s -o /dev/null -w '%{http_code} '
localhost:18080/api/actuator/health ; done ; echo
```

Observe `stack_api_1` and `stack_api_2`, both `(healthy)`; `nslookup` returns, after the server line, **two** addresses for `api` (`10.89.0.3` and `10.89.0.4`); four `200`s.

```
podman compose down -v 2>&1 | tail -2
podman ps -a --format '{{.Names}}'
podman rm -f -t 0 stack_api_2
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman pod ls --format '{{.Name}}' ; podman volume ls --
format '{{.Name}}'
```

Observe the failure of the first `down -v`: `cannot remove container … as it is running` and `"stack_default" has associated containers` — `stack_api_2` stayed, and `stack_db_1` with it. After removing `stack_api_2` by name, the second `down -v` grumbles about what is already gone (`no container with name or ID "stack_web_1" found`) but cleans up the rest: no container, no pod, no volume left.

Explanation. podman-compose does not handle the instances created by `--scale` at `down`. Docker Compose does. Remember the repair: read `podman ps -a`, remove by name, run `down` again.

Step 7 — Building from Compose

```
podman compose build 2>&1 | tail -2
```

Observe `Successfully tagged localhost/api:1.0`: the `build: context: ../05` replayed the lab 05 Dockerfile (from cache), and the image carries the name declared by `image:`.

Explanation. `build` + `image` in the same service: Compose builds **and** names. In CI, `podman compose build` then `podman compose up -d` are enough to test the whole stack.

Clean-up

```
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman network ls
--format '{{.Name}}'
```

Observe that everything is empty (only the `podman` network remains). Keep `~/labo-docker/09/` whole, `db_password.txt` included, and the images `api:1.0`, `postgres:16-alpine`, `nginx:alpine`: lab 10 operates this stack.

What you must be able to state now

- `podman compose up -d` creates network, volume, pod and containers in `depends_on` order: `db` healthy before `api`, `api` healthy before `web`.
- Everything carries the project prefix: `stack_db_1`, `stack_default`, `stack_pgdata`, `pod_stack`.
- `config` shows the merge: automatic override, explicit `-f` that ignores it, `${API_TAG:-1.0}` substituted from `.env` or the shell.
- `down` keeps `stack_pgdata` and the `clients` table; `down -v` deletes them.
- `stop`/`start` of the API → new IP → `502` at nginx, `200` after `restart web`.
- `podman compose exec` without the right `-f` lies (`dev`), `podman exec` tells the truth (`prod`).
- `--scale api=2` gives two addresses to the name `api`; podman-compose cannot remove them: `podman rm -f -t 0 stack_api_2` before `down`.